

E-Commerce React Template - Check24

Vorbereitung

PROJEKT VON NULL STARTEN (Kein Template!)

Schritt 1: React + TypeScript Projekt erstellen

```
npm create vite@latest mein-shop -- --template react-ts  
cd mein-shop
```

Was passiert?

- Erstellt einen neuen Ordner `mein-shop`
 - Installiert React + TypeScript + Vite
 - Grundstruktur ist da
-

Schritt 2: Dependencies installieren

```
# Basis-Pakete installieren  
npm install  
  
# Redux Toolkit + React Redux (State Management)  
npm install @reduxjs/toolkit react-redux  
  
# React Router (falls Routing gebraucht)  
npm install react-router-dom  
  
# JSON Server (Mock API)  
npm install -D json-server  
  
# Concurrently (API + Frontend gleichzeitig starten)  
npm install -D concurrently
```

Alle auf einmal:

```
npm install @reduxjs/toolkit react-redux react-router-dom  
npm install -D json-server concurrently
```

Schritt 3: package.json Scripts ergänzen

Öffne `package.json` und ergänze die Scripts:

```
"scripts": {  
  "dev": "vite",  
  "api": "json-server --watch mock-api/db.json --port 3001",  
  "start": "concurrently \"npm run api\" \"npm run dev\"",  
  "build": "tsc -b && vite build"  
}
```

Schritt 4: Mock-API Ordner erstellen

```
mkdir mock-api
```

Dann `mock-api/db.json` erstellen.

Schritt 5: JSON-Datei vorbereiten

⚠ WICHTIG: JSON-SERVER FORMAT

json-server braucht dieses Format:

```
{  
  "items": [  
    { "id": 1, "name": "Product 1" },  
    { "id": 2, "name": "Product 2" }  
  ]  
}
```

NICHT so (reines Array):

```
[  
  { "id": 1, "name": "Product 1" },  
  { "id": 2, "name": "Product 2" }  
]
```

Wie erkenne ich das Format?

JSON startet mit...	Was ist es?	Was tun?
{	Objekt	☒ Direkt nutzen
[	Array	☐ Wrappen!

Wie wrappe ich ein Array?

Vorher (Array):

```
[  
  { ... },  
  { ... }  
]
```

Nachher (Objekt mit Key):

```
{  
  "items": [  
    { ... },  
    { ... }  
  ]  
}
```

Schritt 6: Ordnerstruktur erstellen

```
# In src/ diese Ordner erstellen:  
mkdir src/store  
mkdir src/features  
mkdir src/features/products  
mkdir src/components
```

Oder manuell in VS Code erstellen.

Schritt 7: Dateien erstellen (in dieser Reihenfolge!)

1. `src/store/store.ts`
2. `src/store/hooks.ts`
3. `src/features/products/types.ts`
4. `src/features/products/product.adapter.ts`
5. `src/features/products/productsSlice.ts`
6. `src/features/products/ProductCard.tsx`
7. `src/features/products/ProductFilters.tsx`

8. `src/features/products/ProductList.tsx`
 9. `src/components/Layout.tsx`
 10. `src/main.tsx` anpassen (Provider!)
 11. `src/App.tsx` anpassen
 12. `src/App.css` anpassen
-

Schritt 8: Starten!

```
npm run start
```



JSON-DATEI OPTIONEN

Option A: Du bekommst eine JSON-Datei

1. Kopiere sie in `mock-api/db.json`
2. Prüfe das Format (Array oder Objekt?)
3. Falls Array → wrappen
4. `product.adapter.ts` anpassen

Option B: API URL gegeben

Falls du eine URL bekommst statt einer Datei:

```
// productsSlice.ts
export const fetchProducts = createAsyncThunk('products/fetch', async () => {
  const res = await fetch('https://die-gegebene-url.com/products');
  const rawData = await res.json();
  return mapProducts(rawData);
});
```

Dann brauchst du KEINEN json-server!

Option C: Statischer Import (kein Server)

Falls json-server nicht funktioniert:

```
// productsSlice.ts
import rawData from '../mock-api/data.json';

export const fetchProducts = createAsyncThunk('products/fetch', async () => {
```

```
// Fake delay
await new Promise(r => setTimeout(r, 100));
return mapProducts(rawData);
});
```

Dann `mock-api/data.json` direkt importieren (kein Server nötig!)

ZUSAMMENFASSUNG: VON 0 BIS FERTIG

1. `npm create vite@latest mein-shop -- --template react-ts`
2. `cd mein-shop`
3. `npm install @reduxjs/toolkit react-redux`
4. `npm install -D json-server concurrently`
5. `package.json` Scripts ergänzen
6. `mock-api/db.json` mit Daten
7. Ordnerstruktur erstellen
8. Dateien erstellen (store → types → adapter → slice → components)
9. `npm run start`
10. FERTIG!

Zeit: ca. 30-45 Minuten wenn du weißt was du tust!

QUICK START - Projekt von GitHub holen (Template nutzen)

```
# 1. Repository klonen
git clone https://github.com/arash-barakzaii/mock.git mein-projekt
cd mein-projekt

# 2. Dependencies installieren
npm install

# 3. Starten (API + Frontend gleichzeitig)
npm run start
```

URLs nach dem Start:

-  Frontend: `http://localhost:5173`
-  API: `http://localhost:3001/items`

ALLE BEFEHLE

Befehl	Was macht es?
--------	---------------

Befehl	Was macht es?
<code>npm install</code>	Installiert alle Pakete
<code>npm run dev</code>	Startet NUR Frontend (Port 5173)
<code>npm run api</code>	Startet NUR API Server (Port 3001)
<code>npm run start</code>	Startet BEIDES gleichzeitig
<code>npm run build</code>	Baut Produktions-Version

📁 ORDNERSTRUKTUR

```
src/
├── store/                                ← Redux Setup
│   ├── store.ts                         ← Zentraler Store
│   └── hooks.ts                         ← Typed Hooks
├── features/products/                  ← Alles zu Produkten
│   ├── types.ts                        ← UI-Modell (IMMER GLEICH!)
│   ├── product.adapter.ts              ← ⚡ HIER MORGEN ANPASSEN!
│   ├── productsSlice.ts                ← Redux State & Logic
│   ├── ProductList.tsx                 ← Hauptansicht
│   ├── ProductCard.tsx                 ← Eine Produkt-Karte
│   ├── ProductFilters.tsx              ← Such- & Filterleiste
│   └── ProductDetail.tsx               ← Detail-Modal
├── components/
│   └── Layout.tsx                      ← Header & Layout
├── App.tsx                             ← Haupt-App
├── App.css                             ← Alle Styles
└── main.tsx                            ← Einstiegspunkt

mock-api/
└── items.json                          ← Mock-Daten (db.json)
```

📄 JEDE DATEI ERKLÄRT

1 `store/store.ts` - Das Gehirn

```
import { configureStore } from '@reduxjs/toolkit';
import productsReducer from '../features/products/productsSlice';

export const store = configureStore({
```

```
    reducer: {
      products: productsReducer,
    },
  });

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

Was macht es?

- `configureStore()` → Erstellt den zentralen Datenspeicher
- `reducer` → "Abteilungen" im Store (products, cart, etc.)
- `RootState` → TypeScript Type für den ganzen State
- `AppDispatch` → TypeScript Type für dispatch

Wann ändern?

- Neues Feature (z.B. Cart) → Neuen Reducer hinzufügen:

```
reducer: {
  products: productsReducer,
  cart: cartReducer, // ← NEU
}
```

2 store/hooks.ts - Typed Hooks

```
import type { TypedUseSelectorHook } from 'react-redux';
import { useDispatch, useSelector } from 'react-redux';
import type { RootState, AppDispatch } from './store';

export const useAppDispatch = () => useDispatch<AppDispatch>();
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector;
```

Was macht es?

- `useAppSelector` → Daten aus Store lesen (mit TypeScript!)
- `useAppDispatch` → Aktionen auslösen (mit TypeScript!)

Wann ändern? → NIEMALS! Bleibt immer gleich.

3 features/products/types.ts - Das Datenmodell

```
export interface UiProduct {
  id: string;
```

```

    title: string;
    price: number;
    imageUrl?: string;
    tags: string[];
    category?: string;
    description?: string;
  }

  export interface ProductsState {
    items: UiProduct[];
    loading: boolean;
    error: string | null;
    searchTerm: string;
    sortBy: 'title' | 'price-asc' | 'price-desc';
    tagFilter: string | null;
    selectedProduct: UiProduct | null;
  }

```

Was macht es?

- `UiProduct` → So sieht ein Produkt für die UI aus
- `ProductsState` → Alle Daten die gespeichert werden

Wann ändern?

- Neue Filter → `ProductsState` erweitern
- Neues Feld anzeigen → `UiProduct` erweitern

4 features/products/product.adapter.ts - ⚡ DER ÜBERSETZER

```

import type { UiProduct } from './types';

export function mapToUiProduct(raw: any): UiProduct {
  return {
    id: String(raw.slug ?? raw.id ?? raw.imageHash),
    title: raw.name ?? raw.title ?? 'Unknown',
    price: Number(raw.price ?? 0),
    imageUrl: raw.imageUrl ?? raw.image,
    tags: raw.tags ?? [],
    category: raw.itemType ?? raw.category,
    description: raw.description,
  };
}

export function mapProducts(rawData: any[]): UiProduct[] {
  return rawData.map(mapToUiProduct);
}

```

Was macht es?

- Übersetzt die rohe JSON in unser UI-Format
- `raw.name` → `title`
- `raw.itemType` → `category`

⚡ **MORGEN: Diese Datei anpassen wenn neue JSON kommt!**

Beispiel: Neue JSON hat andere Feldnamen:

```
// Vorher (aktuelle JSON)
title: raw.name,
category: raw.itemType,

// Nachher (neue JSON)
title: raw.produktName,
category: raw.kategorie,
```

5 features/products/productsSlice.ts - Die Logik

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import type { PayloadAction } from '@reduxjs/toolkit';
import type { UiProduct, ProductsState } from './types';
import { mapProducts } from './product.adapter';

const API_URL = 'http://localhost:3001';

// ASYNC THUNK - Holt Daten vom Server
export const fetchProducts = createAsyncThunk('products/fetch', async () => {
  const res = await fetch(`${API_URL}/items`);
  const rawData = await res.json();
  return mapProducts(rawData); // ← Adapter!
});

// INITIAL STATE - Startwerte
const initialState: ProductsState = {
  items: [],
  loading: false,
  error: null,
  searchTerm: '',
  sortBy: 'title',
  tagFilter: null,
  selectedProduct: null,
};

// SLICE - State + Actions
const productsSlice = createSlice({
  name: 'products',
  initialState,
  reducers: {
    // Synchronische Aktionen
```

```
    setSearchTerm: (state, action: PayloadAction<string>) => {
      state.searchTerm = action.payload;
    },
    setSortBy: (state, action: PayloadAction<'title' | 'price-asc' | 'price-desc'>) => {
      state.sortBy = action.payload;
    },
    setTagFilter: (state, action: PayloadAction<string | null>) => {
      state.tagFilter = action.payload;
    },
    setSelectedProduct: (state, action: PayloadAction<UiProduct | null>) => {
      state.selectedProduct = action.payload;
    },
  },
  extraReducers: (builder) => {
    // Async Aktionen
    builder
      .addCase(fetchProducts.pending, (state) => {
        state.loading = true;
      })
      .addCase(fetchProducts.fulfilled, (state, action) => {
        state.loading = false;
        state.items = action.payload;
      })
      .addCase(fetchProducts.rejected, (state) => {
        state.loading = false;
        state.error = 'Failed to load';
      });
  },
});

export const { setSearchTerm, setSortBy, setTagFilter, setSelectedProduct } =
  productsSlice.actions;
export default productsSlice.reducer;
```

Wichtige Teile:

Teil	Was macht es?
<code>createAsyncThunk</code>	Für API-Calls (pending → fulfilled → rejected)
<code>initialState</code>	Startwerte für alles
<code>reducers</code>	Synchrone Aktionen (Filter ändern, etc.)
<code>extraReducers</code>	Reaktion auf Async-Aktionen

Wann ändern?

- Neuer Filter → Neuen Reducer hinzufügen + `initialState` erweitern
- Anderer API-Endpunkt → `fetch()` URL ändern

6 features/products/ProductList.tsx - Hauptansicht

```
export const ProductList = () => {
  const dispatch = useAppDispatch();
  const { items, loading, searchTerm, sortBy, tagFilter, selectedProduct } =
    useAppSelector(s => s.products);
  const [limit, setLimit] = useState(12);

  // Beim Start: Daten laden
  useEffect(() => {
    dispatch(fetchProducts());
  }, [dispatch]);

  // Filter & Sort (gecached!)
  const filtered = useMemo(() => {
    let result = [...items];

    // Tag Filter
    if (tagFilter) {
      result = result.filter(p => p.tags.includes(tagFilter));
    }

    // Suche
    if (searchTerm) {
      result = result.filter(p =>
        p.title.toLowerCase().includes(searchTerm.toLowerCase())
      );
    }

    // Sortierung
    result.sort((a, b) => {
      if (sortBy === 'title') return a.title.localeCompare(b.title);
      if (sortBy === 'price-asc') return a.price - b.price;
      if (sortBy === 'price-desc') return b.price - a.price;
      return 0;
    });

    return result;
  }, [items, searchTerm, sortBy, tagFilter]);

  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error!</div>;

  return (
    <div>
      <ProductFilters />
      {filtered.slice(0, limit).map(p => (
        <ProductCard
          key={p.id}
          product={p}
          onClick={() => dispatch(setSelectedProduct(p))}
        />
      ))}
    </div>
  );
}
```

```

    )))
    { /* Load More Button */ }
    { /* Detail Modal */ }
  </div>
);
};

```

Wichtige Hooks:

Hook	Was macht es?
<code>useEffect</code>	Führt Code beim Mount aus (Daten laden)
<code>useMemo</code>	Cached Berechnung (nur neu wenn Deps ändern)
<code>useState</code>	Lokaler State (limit für Load More)
<code>useAppSelector</code>	Daten aus Redux Store lesen
<code>useAppDispatch</code>	Actions an Store senden

Wann ändern?

- Andere Sortierung → `useMemo` anpassen
- Anderer Filter → `useMemo` anpassen

7 features/products/ProductFilters.tsx - Filterleiste

```

export const ProductFilters = () => {
  const dispatch = useAppDispatch();
  const { searchTerm, sortBy, tagFilter, items } = useAppSelector(s =>
s.products);

  // Alle Tags aus Produkten extrahieren
  const allTags = useMemo(() => {
    const tags = new Set<string>();
    items.forEach(p => p.tags.forEach(t => tags.add(t)));
    return Array.from(tags).sort().slice(0, 20);
  }, [items]);

  return (
    <div className="filters">
      <input
        value={searchTerm}
        onChange={(e) => dispatch(setSearchTerm(e.target.value))}
        placeholder="Search..."
      />

      <select
        value={tagFilter || ''}
        onChange={(e) => dispatch(setTagFilter(e.target.value || null))}

```

```

    >
    <option value="">All Tags</option>
    {allTags.map(tag => <option key={tag} value={tag}>{tag}</option>)}
  </select>

  <select
    value={sortBy}
    onChange={(e) => dispatch(setSortBy(e.target.value as any))}
  >
    <option value="title">Name A-Z</option>
    <option value="price-asc">Price ↑</option>
    <option value="price-desc">Price ↓</option>
  </select>
</div>
);
};

```

Der Datenfluss:

```

User tippt "Mug"
  ↓
onChange → dispatch(setSearchTerm("Mug"))
  ↓
productsSlice.reducers.setSearchTerm()
  ↓
state.searchTerm = "Mug"
  ↓
ProductList re-rendert → useMemo filtert
  ↓
Gefilterte Liste wird angezeigt

```

Wann ändern?

- Neuer Filter → Neues `<select>` + neue Action im Slice

8 features/products/ProductCard.tsx - Eine Karte

```

export const ProductCard = ({ product, onClick }: Props) => {
  const [imgError, setImgError] = useState(false);

  return (
    <div className="product-card" onClick={onClick}>
      {!imgError && product.imageUrl ? (
        <img
          src={product.imageUrl}
          onError={() => setImgError(true)} // Fallback!
        />
      ) : (

```

```
    <div className="placeholder">🖼️</div>
  })
  <span>{product.category}</span>
  <h3>{product.title}</h3>
  <p>€{product.price.toFixed(2)}</p>
  <div className="tags">
    {product.tags.map(tag => <span key={tag}>{tag}</span>)}
  </div>
</div>
);
};
```

Wann ändern?

- Andere Felder anzeigen → JSX anpassen (aber NUR wenn types.ts auch geändert!)

9 main.tsx - Der Startpunkt

```
import { Provider } from 'react-redux';
import { store } from './store/store';

createRoot(document.getElementById('root')!).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

WICHTIG: <Provider store={store}> macht Redux für die ganze App verfügbar!



WICHTIGE FUNKTIONEN ÜBERSICHT

Funktion	Was macht es?	Import von
configureStore()	Erstellt Redux Store	@reduxjs/toolkit
createSlice()	State + Reducers	@reduxjs/toolkit
createAsyncThunk()	Async API-Calls	@reduxjs/toolkit
useAppSelector()	Daten lesen	store/hooks
useAppDispatch()	Actions senden	store/hooks
useEffect()	Side Effects	react
useMemo()	Cached Berechnung	react
useState()	Lokaler State	react



DER KOMPLETTE DATENFLUSS

1. App startet
↓
2. main.tsx rendert `<Provider store={store}>`
↓
3. ProductList mounted → useEffect läuft
↓
4. `dispatch(fetchProducts())` wird aufgerufen
↓
5. Async Thunk: pending → loading = true
↓
6. `fetch()` holt Daten von `localhost:3001/items`
↓
7. `mapProducts()` wandelt JSON → `UiProduct[]`
↓
8. fulfilled → items = gemappte Daten
↓
9. ProductList re-rendert
↓
10. `useMemo` filtert & sortiert
↓
11. ProductCards werden angezeigt

⚡ MORGEN: WAS ANPASSEN?

Schritt 1: JSON bekommen & anschauen

```
{
  "produkte": [
    {
      "artikelId": 1,
      "produktName": "Laptop",
      "kosten": 999,
      "bild": "https://..."
    }
  ]
}
```

Schritt 2: JSON in `mock-api/db.json` kopieren

Falls Array: Wrappen in Objekt:

```
{
  "items": [
    // ... dein Array hier
  ]
}
```

Schritt 3: `product.adapter.ts` anpassen

```
export function mapToUiProduct(raw: any): UiProduct {
  return {
    id: String(raw.artikelId),      // ← Neuer Feldname
    title: raw.produktName,        // ← Neuer Feldname
    price: Number(raw.kosten),     // ← Neuer Feldname
    imageUrl: raw.bild,            // ← Neuer Feldname
    tags: raw.labels ?? [],
    category: raw.kategorie,
    description: raw.beschreibung,
  };
}
```

Schritt 4: API-Endpunkt prüfen

In `productsSlice.ts`:

```
const res = await fetch(`${API_URL}/items`); // oder /produkte?
```

Schritt 5: FERTIG! 🎉

NEW NEUE FEATURES HINZUFÜGEN

Neuer Filter (z.B. Kategorie)?

1. `types.ts` - State erweitern:

```
export interface ProductsState {
  // ...
  categoryFilter: string | null; // ← NEU
}
```

2. `productsSlice.ts` - Reducer hinzufügen:

```
// initialState
categoryFilter: null,

// reducers
setCategoryFilter: (state, action: PayloadAction<string | null>) => {
  state.categoryFilter = action.payload;
},

// Export
export const { ..., setCategoryFilter } = productsSlice.actions;
```

3. ProductFilters.tsx - UI hinzufügen:

```
<select
  value={categoryFilter || ''}
  onChange={(e) => dispatch(setCategoryFilter(e.target.value || null))}
>
  <option value="">All Categories</option>
  { /* ... */ }
</select>
```

4. ProductList.tsx - useMemo erweitern:

```
if (categoryFilter) {
  result = result.filter(p => p.category === categoryFilter);
}
```

Favoriten hinzufügen?

1. Neues Slice erstellen: features/favorites/favoritesSlice.ts

2. In store.ts hinzufügen:

```
reducer: {
  products: productsReducer,
  favorites: favoritesReducer, // ← NEU
}
```

3. In ProductCard Button hinzufügen

Warenkorb hinzufügen?

1. Neues Slice: `features/cart/cartSlice.ts`
2. In `store.ts` hinzufügen
3. Cart-Component erstellen
4. In ProductCard "Add to Cart" Button



SORTIERUNG ERKLÄRT

```
result.sort((a, b) => {  
  // Aufsteigend (klein → groß): a - b  
  if (sortBy === 'price-asc') return a.price - b.price;  
  
  // Absteigend (groß → klein): b - a  
  if (sortBy === 'price-desc') return b.price - a.price;  
  
  // Alphabetisch  
  if (sortBy === 'title') return a.title.localeCompare(b.title);  
  
  return 0;  
});
```



HÄUFIGE FEHLER & LÖSUNGEN

Problem	Lösung
<code>import type</code> Error	Types mit <code>import type { X }</code> importieren
JSON-Server geht nicht	JSON muss <code>{ "key": [...] }</code> Format haben
Redux funktioniert nicht	<code><Provider store={store}></code> in main.tsx?
Falscher Ordner	<code>cd projektname</code> nicht vergessen
Bilder laden nicht	<code>onError</code> Fallback in img-Tag
API 404	Endpoint prüfen: <code>/items</code> oder <code>/products</code> ?



CHECKLISTE MORGEN

- ☐ JSON anschauen → Struktur verstehen
- ☐ JSON in `mock-api/db.json` (mit `{ "items": [...] }`)
- ☐ `product.adapter.ts` anpassen
- ☐ API-Endpoint in `productsSlice.ts` prüfen

- ☐ `npm install && npm run start`
- ☐ App läuft → Los coden!

PRIORITÄTEN AM TAG

1. ☒ Projekt aufsetzen (5 min)
2. ☒ JSON + Adapter anpassen (10 min)
3. ☒ Sicherstellen Filter/Sort funktioniert (10 min)
4. ☒ Detail-Ansicht funktioniert (10 min)
5. ★ Extras wenn Zeit: Favoriten, Cart, besseres Styling

Viel Erfolg morgen! 🙌 🌀